

Supabase & PostgreSQL Best Practices

SECTION 06-operations-it

TYPE document

STATUS **active**

OWNER DevOps & Backend

UPDATED 2026-09-05

This document captures best practices and conventions for using Supabase and PostgreSQL across the Fly GACA family, based on current usage in `ay2m/FlyGACA`, `ay2m/Captain-Adel`, and `ay2m/FlyGACA-ios`.

Overview

Fly GACA uses **PostgreSQL on Supabase** (Cloud SQL on Cloud Run in production) for the primary application datastore and vector embeddings for the Captain Adel RAG pipeline. The Stack:

- **Primary backend:** Express 5 API (Cloud Run, `me-central2` Dammam region)
- **Database:** Cloud SQL PostgreSQL (Supabase in dev/local, Cloud SQL in production)
- **Vector store:** PostgreSQL with `pgvector` extension for GACAR corpus embeddings
- **Migrations:** Forward-only, applied via `scripts/migrate.mjs`
- **Data residency:** `me-central2` Dammam only — never `me-central1` (Doha, Qatar)

1. Database Architecture & Design

1.1 Core Principles

- **Schema-first development:** Migrations are the source of truth. Every schema change goes through version control before production.
- **Forward-only migrations:** No `ALTER TABLE DROP` or destructive changes in existing migrations. New requirements mean new migrations.
- **Immutable audit trail:** All data mutations include `created_at` and `updated_at` timestamps; sensitive tables carry an audit log.
- **Parameterized queries:** Always. Never concatenate user input into SQL.
- **TypeScript types mirror database schema:** Server code generates or hand-writes types that match columns exactly.

1.2 Connection Management

Connection pooling in production:

```
// server/src/db.ts
const pool = new pg.Pool({
  connectionString: config.db.url, // Cloud SQL unix socket:
  postgresql://user:pass@db?host=/cloudsql/PROJECT:REGION:INSTANCE
  max: config.db.poolMax,           // 5 (low for horizontal scaling on Cloud Run)
  idleTimeoutMillis: 10_000,         // 10s (Cloud Run freezes idle instances)
  connectionTimeoutMillis: 10_000,
  ssl: { rejectUnauthorized: false } // Non-localhost, non-cloudsql connections
});
```

Critical: Cloud SQL connections use **Unix sockets** (`/cloudsql/PROJECT:REGION:INSTANCE`), not network IPs. This:

- Eliminates the need for IP allowlisting
- Removes the Cloud SQL proxy sidecar
- Keeps connections inside the GCP private network

Error handling: Idle client deaths (failovers, maintenance) are logged but not fatal — the next query gets a fresh connection:

```
pool.on("error", (err) => {
  console.error("Idle Postgres client died; connection discarded:", err);
});
```

1.3 Data Residency & Compliance

Golden rule: All learner and operational data stays in `me-central2` (Dammam, Saudi Arabia).

- **Production Cloud SQL instance:** Pinned to `me-central2`. The deploy script hard-fails if `REGION ≠ me-central2`.
- **Never `me-central1`:** That is Doha, Qatar — a different country. Not PDPL-compliant.
- **Backup encryption:** Daily encrypted backups, same region.
- **Cross-region inference:** Gemini API calls leave the Kingdom (Google's endpoint is in the US). This is documented risk, not hidden.

2. Migrations & Schema Evolution

2.1 Migration Workflow

Migrations live in `server/migrations/*.sql` and are applied via `server/scripts/migrate.mjs`:

```
node --env-file=.env server/scripts/migrate.mjs
```

The runner:

1. Connects to the database

2. Creates `schema_migrations` table (if missing) to track applied migrations
3. Reads every `.sql` file in filename order
4. Applies each one not yet in the tracker, inside its own transaction
5. Commits or rolls back atomically

Naming convention: `NNNN_description.sql` (e.g., `0001_init.sql`, `0002_add_apple_oauth.sql`).

2.2 Writing Safe Migrations

Do:

-  Add columns with defaults: `ALTER TABLE users ADD COLUMN email_verified boolean NOT NULL DEFAULT false;`
-  Create new indexes: `CREATE INDEX idx_name ON table (column);`
-  Add constraints: `ALTER TABLE users ADD CONSTRAINT email_not_empty CHECK (email ≠ '');`
-  Rename columns carefully: `ALTER TABLE users RENAME COLUMN old_name TO new_name;`

Don't:

-  Remove columns or tables in the same migration as data migration — two separate files
-  Lock tables for long operations — Cloud Run instances have 60s request timeouts
-  Create expensive indexes in the hot path — background jobs only
-  Assume `pg_dump` will be manually restored — test every migration with the script

2.3 Example Migration

```
-- 0004_billing_webhooks.sql
-- Add webhook tracking for payment provider callbacks (Moyasar).

CREATE TABLE IF NOT EXISTS webhook_events (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  provider    text NOT NULL,
  event_type  text NOT NULL,
  payload     jsonb NOT NULL,
  processed   boolean NOT NULL DEFAULT false,
  error_msg   text,
  received_at timestamptz NOT NULL DEFAULT now(),
  processed_at timestamptz
);

CREATE INDEX webhook_events_processed_idx
  ON webhook_events (provider, processed)
  WHERE NOT processed;
```

3. Vector Embeddings & RAG

3.1 The Regulation Chunks Table

The Captain Adel RAG pipeline uses `pgvector` to store and search GACAR regulatory embeddings:

```
-- supabase/migrations/0001_regulations_embeddings.sql

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS public.regulation_chunks (
  id          text PRIMARY key,          -- stable: "part-91::section-5"
  slug        text NOT NULL,            -- "part-91"
  part_num    integer NOT NULL,          -- 91
  section     text NOT NULL,            -- heading/section label
  content     text NOT NULL,            -- the actual text
  content_hash text NOT NULL,            -- sha256; skip re-upsert if unchanged
  embedding   vector(1536) NOT NULL,     -- OpenAI text-embedding-3-small
  metadata    jsonb NOT NULL DEFAULT '{}', -- future extensions
  updated_at  timestampz NOT NULL DEFAULT now()
);

-- Approximate nearest-neighbor search. Tune `lists` to ~sqrt(row_count) as corpus grows.
CREATE INDEX regulation_chunks_embedding_idx
  ON public.regulation_chunks
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);

-- Vector similarity RPC for grounded retrieval.
CREATE OR REPLACE FUNCTION public.match_regulations(
  query_embedding vector(1536),
  match_count int DEFAULT 8,
  similarity_threshold float DEFAULT 0.0
)
RETURNS TABLE (
  id text, slug text, part_num integer, section text, content text, metadata jsonb,
  similarity float
)
LANGUAGE SQL STABLE
AS $$
  SELECT
    c.id, c.slug, c.part_num, c.section, c.content, c.metadata,
    1 - (c.embedding <=> query_embedding) AS similarity
  FROM public.regulation_chunks c
  WHERE 1 - (c.embedding <=> query_embedding) >= similarity_threshold
  ORDER BY c.embedding <=> query_embedding
  LIMIT match_count;
$$;
```

3.2 Upserting Embeddings

The GitHub Action `docs-parser` runs on each content commit and:

1. Chunks GACAR Parts by section (Markdown splitter)
2. Embeds each chunk with OpenAI `text-embedding-3-small` (1536 dims)
3. Upserts rows into `regulation_chunks` (skip if `content_hash` unchanged)

Tuning:

- `lists = 100` works for ~10,000 chunks. As the corpus grows, tune to `~sqrt(rows)`.
- After large upserts, run `ANALYZE public.regulation_chunks;` to update statistics.
- Vector similarity ranges [0, 1]; a threshold of `0.5` means at least 50% cosine similarity.

3.3 Querying Embeddings

From the server (Captain Adel RAG):

```
const result = await db.query(
  `SELECT * FROM match_regulations($1, $2, $3)`,
  [queryEmbedding, 8, 0.5] // embedding, match_count, similarity_threshold
);
```

Returns closest regulatory chunks with their Part number, section, and exact content for citation.

4. User & Auth Data

4.1 Users Table

```
CREATE TABLE IF NOT EXISTS users (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email       citext NOT NULL UNIQUE,
  email_verified boolean NOT NULL DEFAULT false,
  password_hash text,
  display_name text NOT NULL DEFAULT '',
  google_sub  text UNIQUE,          -- OAuth: Google
  apple_sub   text UNIQUE,          -- OAuth: Apple
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);
```

Extensions in use:

- `citext`: Case-insensitive email comparison (UNIQUE constraint is case-insensitive)
- `pgcrypto`: For `gen_random_uuid()` and secure token generation

PDPL compliance:

- Email, display name, and progress only. No passport, address, voice, biometrics.
- Account deletion triggers cascade delete of all related data (`ON DELETE CASCADE`).
- Consent timestamp tracked separately (see `org_seats.pdpL_consent`).

4.2 Auth Tokens

```
CREATE TABLE IF NOT EXISTS auth_tokens (  
  digest      text PRIMARY KEY,          -- sha256(token)  
  user_id     uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  kind        text NOT NULL CHECK (kind IN ('verify', 'reset')),  
  expires_at  timestampz NOT NULL,  
  used_at     timestampz,  
  created_at  timestampz NOT NULL DEFAULT now()  
);  
CREATE INDEX auth_tokens_user_idx ON auth_tokens (user_id, kind);
```

Token security:

- Tokens are **hashed** (never stored plaintext).
- Expiry is enforced in queries (`WHERE expires_at > now()`).
- One-time use is tracked (`used_at`).
- Tokens are never returned as cookies — JWT is the session mechanism.

5. Session & JWT Management

Sessions are **HttpOnly** JWTs, not database-backed:

```
// server/src/auth-core.ts  
const token = jwt.sign(  
  { sub: userId, email: userEmail },  
  sessionSecret,  
  { expiresIn: "30d", algorithm: "HS256" }  
);  
  
// Set as HttpOnly cookie, never localStorage  
response.cookie("fg_session", token, {  
  httpOnly: true,  
  secure: process.env.NODE_ENV === "production",  
  sameSite: "Lax",  
  domain: config.sessionCookieDomain,  
  maxAge: config.sessionTTL * 24 * 60 * 60 * 1000  
});
```

Key properties:

- **HttpOnly**: JavaScript cannot access it (XSS defense)
- **Secure flag**: HTTPS only in production
- **SameSite=Lax**: CSRF defense (cross-site requests don't include the cookie)
- **Rotation**: Secrets rotate via Secret Manager; old tokens become invalid at `exp` time

6. Data Pipeline & PDPL

6.1 Learner Data Tables

```
CREATE TABLE study_progress (  
  user_id      uuid PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,  
  summary      jsonb NOT NULL DEFAULT '{}', -- SRS state per pack  
  updated_at   timestampz NOT NULL DEFAULT now()  
);  
  
CREATE TABLE flight_hours (  
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id      uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  date         date NOT NULL,  
  duration     integer NOT NULL,           -- minutes  
  aircraft     text NOT NULL,             -- "Cessna 172"  
  instructor   text NOT NULL,             -- name  
  module       text NOT NULL,             -- "ppl", "elpt", etc.  
  notes       text DEFAULT '',            -- instructor feedback (anonymized in  
reports)  
  created_at   timestampz NOT NULL DEFAULT now()  
);  
CREATE INDEX flight_hours_user_date_idx ON flight_hours (user_id, date);
```

6.2 PDPL & Anonymization

For aggregate reporting (health scoring, cohort analytics):

- Individual flights are anonymized (remove name, date details)
- Only totals and counts are exposed to business analytics
- Deletion/right-to-be-forgotten queries clear or anonymize rows immediately
- Data retention policy: 2–7 years post-engagement, then deletion

6.3 Audit Trail

```
CREATE TABLE IF NOT EXISTS audit_log (  
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id      uuid,  
  action       text NOT NULL,             -- "login", "purchase", "data_export",  
"delete"  
  resource     text NOT NULL,             -- table/id  
  old_val      jsonb,  
  new_val      jsonb,  
  timestamp    timestampz NOT NULL DEFAULT now()  
);
```

All mutations to sensitive tables (users, entitlements, flight_hours) trigger an audit log entry.

7. Transactions & Consistency

7.1 Serializable Transactions

The `tx()` helper enforces **serializable** semantics via `SELECT ... FOR UPDATE`:

```
// server/src/db.ts
export async function tx<T>(  
  fn: (client: pg.PoolClient) => Promise<T>  
) : Promise<T> {  
  const client = await getPool().connect();  
  try {  
    await client.query("BEGIN");  
    const out = await fn(client);  
    await client.query("COMMIT");  
    return out;  
  } catch (err) {  
    await client.query("ROLLBACK").catch(() => {});  
    throw err;  
  } finally {  
    client.release();  
  }  
}
```

Use this for:

- Multi-step mutations (e.g., debit credit, credit account in one atomic step)
- Conflict detection (e.g., duplicate flight hour records)

Example:

```
await tx(async (client) => {  
  const row = await client.query(  
    "SELECT * FROM entitlements WHERE user_id = $1 FOR UPDATE",  
    [userId]  
  );  
  // Guaranteed no concurrent update can change this row while we hold the lock  
  if (row.rows[0].credits ≥ questionsNeeded) {  
    await client.query(  
      "UPDATE entitlements SET credits = credits - $1 WHERE user_id = $2",  
      [questionsNeeded, userId]  
    );  
  }  
});
```

7.2 Isolation Levels

PostgreSQL default is `READ COMMITTED`. For critical operations, raise to `SERIALIZABLE`:

```
BEGIN ISOLATION LEVEL SERIALIZABLE;  
  -- Your statements  
COMMIT;
```


Cloud Run's 60-second timeout is tight for serializable isolation on hot tables. Keep transactions short.

8. Indexes & Performance

8.1 Index Strategy

Create indexes for:

- ☒ Foreign key columns (`user_id` , `org_id`)
- ☒ Filter columns in `WHERE` clauses (`processed` , `kind`)
- ☒ Sort columns in `ORDER BY` (especially with `LIMIT`)
- ☒ Vector similarity (IVFFlat for approximate nearest neighbor)

Don't create indexes for:

- ☒ Columns you never filter on
- ☒ Low-cardinality enums (unless very large table)
- ☒ Large TEXT columns unless used with `LIKE 'prefix%'` (not substring search)

Example compound index:

```
CREATE INDEX webhook_events_processed_idx
ON webhook_events (provider, processed)
WHERE NOT processed; -- Partial index: only unprocessed events
```

8.2 Index Maintenance

After large bulk inserts or deletes, rebuild statistics:

```
ANALYZE public.regulation_chunks;
REINDEX TABLE public.regulation_chunks;
```

9. Production Deployment Checklist

Before pushing a migration to production:

- ☐ Migration is tested locally (`npm run dev` + manual test)
- ☐ Migration is forward-only (no destructive drops)
- ☐ Any new columns have defaults to avoid populating NULL on large tables
- ☐ Indexes are created after data population (not before)
- ☐ All FOREIGN KEY constraints reference existing tables
- ☐ PDPL rules are followed (no PII in audit logs, no plaintext passwords)

- ☐ Data residency is `me-central2` (no multi-region queries)

Deployment order:

1. Push migration to main branch
2. CI tests the migration against a staging database
3. Cloud Run deploy runs `node scripts/migrate.mjs` on startup
4. Health check (`/healthz`) verifies connectivity before traffic is routed

10. Monitoring & Debugging

10.1 Health Checks

The server exposes `/healthz` — a cheap liveness probe:

```
app.get("/healthz", async (req, res) => {
  const ok = await ping();
  res.status(ok ? 200 : 503).json({ ok });
});
```

This queries the database and returns 200 only if the connection is live. Cloud Run uses this for rolling deployments.

10.2 Query Performance

Enable query logging (local dev only):

```
export DEBUG_QUERIES=1 # server/src/db.ts checks this flag
```

Slow queries are logged with their duration. Cloud SQL monitoring shows:

- Connection count
- CPU and memory
- Query throughput
- Replication lag (if applicable)

10.3 Common Issues

Issue	Symptom	Fix
Idle connection dies	"connection discarded" in logs	Expected, next query gets fresh connection
Migration stuck	No output for >1min	Check <code>schema_migrations</code> table; manually mark as applied if truly safe
Vector query slow	>500ms response	Check if <code>ANALYZE</code> has run; tune <code>lists</code> parameter

Issue	Symptom	Fix
PDPL consent not tracked	User can query but no <code>pdpL_consent</code> row	Run migration 0003 if missing
Cross-region latency	Queries >100ms from app	Confirm <code>host=/cloudsql/me-central2:...</code> in <code>DATABASE_URL</code>

11. Local Development

11.1 Docker Postgres Setup

```
# Start local Postgres (once)
docker run --name flygaca-postgres \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=flygaca \
  -p 5432:5432 \
  postgres:16

# Apply migrations
DATABASE_URL=postgresql://postgres:postgres@localhost/flygaca \
  node server/scripts/migrate.mjs
```

11.2 Seeding for Development

Create `server/scripts/seed.mjs`:

```
import db from '../src/db.ts';

const testUser = {
  email: 'test@example.com',
  email_verified: true,
  display_name: 'Test Pilot'
};

await db.query(
  `INSERT INTO users (email, email_verified, display_name)
  VALUES ($1, $2, $3) ON CONFLICT DO NOTHING`,
  [testUser.email, testUser.email_verified, testUser.display_name]
);

console.log('Seed complete');
```

12. References

- PostgreSQL Documentation: <https://www.postgresql.org/docs/16/>

- pgvector Extension: <https://github.com/pgvector/pgvector>
 - pg (Node driver): <https://node-postgres.com/>
 - PDPL (Saudi Data Privacy Law): See `ay2m/Office/04-compliance-ksa/`
 - Production Deploy Runbook: `ay2m/FLyGACA/docs/RUNBOOK-deploy.md`
-

Document Info

Owner: Backend & DevOps team

Last Updated: 2026-09-05

Version: 1.0 (initial)

References: `ay2m/FLyGACA` (main usage), `ay2m/Captain-AdeL` (RAG), `ay2m/FLyGACA-ios` (client, no direct Supabase usage)